TECHNICAL PROJECT REPORT

E-Commerce & Inventory Management System

A Pure Java Swing & MySQL Desktop Application for Retail Operations and Order Fulfillment

Submitted as the final capstone deliverable for Project 1 under the Qveto Technology Java Internship program. This report documents the software architecture, core customer and administrator modules, relational database design, connection pooling, security controls, automated verification tests, and execution instructions.

Project Title	E-Commerce & Inventory Management System (Project 1)
Internship Program	Qveto Technology Java Internship
Submitted By	Chetan Reddy (GitHub: Chetanreddysai)
Repository Owner / Collaborator	Harsh Verma (GitHub: Hack44)
Technology Stack	Java (JDK 17+ / JDK 21), Java Swing, AWT, MySQL 8.x, JDBC
Build & Execution	Apache Maven (pom.xml) and Windows Batch Scripts (run.bat, test.bat)
Repository & Pull Request	https://github.com/Hack44/Qveto-java-team-project (PR #14)
Verification Status	19 of 19 Automated Integrity & Integration Tests Passed (100%)
Submission Date	November 2026

Abstract

Modern retail businesses require tight coordination between customer-facing storefronts and backend store operations. Managing catalog stock, promotional discounts, shipping addresses, tax invoices, consignment tracking, product returns, and sales analytics in separate spreadsheets often causes inventory mismatches and delayed order fulfillment.

This project implements a desktop E-Commerce and Inventory Management System written completely in Java (JDK 17+/21) using Java Swing for the graphical interface and MySQL 8.x with JDBC for persistence. The application separates presentation screens, business service stores, and SQL data access objects (DAOs) across 61 Java source and test files. It includes custom connection pooling (DBConnectionPool) using dynamic proxies, an exponential-backoff retry policy (DatabaseRetryPolicy), a custom pure-Java Adobe PDF 1.4 invoice generator (PdfInvoiceGenerator), SHA-256 password hashing, XSS input sanitization, and a Dual-Engine fallback mode that lets all workflows operate in memory if MySQL is offline. Across two standalone verification suites, all 19 automated tests pass cleanly.

Report Contents

Chapter	Section Title
Chapter 1	Introduction & Problem Statement
Chapter 2	System Architecture & Dual-Engine Design
Chapter 3	Technology Stack & Environment
Chapter 4	Customer Portal Modules
Chapter 5	Administrator Command Center Modules
Chapter 6	Application User Interface Showcase
Chapter 7	Database Design & Persistence Infrastructure
Chapter 8	Security Architecture & Input Validation
Chapter 9	Testing & Quality Verification
Chapter 10	Installation & Execution Guide
Chapter 11	Conclusion & Future Scope

Chapter 1: Introduction & Problem Statement

1.1 Background & Problem Statement

Small and mid-sized retail operations often struggle when customer orders, stock counts, invoicing, and financial summaries are tracked manually or in disconnected utilities. When stock quantities are not deducted atomically at checkout or restored after cancellations and returns, overselling and inaccurate sales reporting occur frequently. Similarly, relying on external third-party report engines or fragile database setups makes desktop deployment and offline evaluation difficult.

1.2 Project Objectives

The main engineering goals for Project 1 are:

- Build a dual-role desktop software application supporting both Customers and Administrators in a unified Java Swing interface.
- Maintain clean modular separation across domain models, business service stores, JDBC Data Access Objects (DAOs), and Swing UI components.
- Ensure inventory integrity during checkout, order cancellation, batch restocking, and approved product returns.
- Generate formal vector PDF tax invoices and standard RFC 4180 CSV exports directly in Java without external reporting libraries.
- Support a Dual-Engine architecture that connects cleanly to MySQL when configured while falling back automatically to pre-seeded in-memory demo data when MySQL is unavailable.

1.3 System Scope

The system encompasses user registration and SHA-256 authentication, catalog filtering and sorting, wishlist curation, coupon discount evaluation, multi-address management, checkout with simulated payment settlements, order fulfillment steppers, consignment tracking (AWB), customer reviews, return and refund approvals, low-stock procurement restocking, date-range KPI analytics, and an immutable operational audit journal.

Chapter 2: System Architecture & Dual-Engine Design

2.1 Four-Tier Architecture

The codebase under com.qveto.ecommerce is organized into four clean, distinct architectural layers comprising 61 Java files (11 domain models, 8 JDBC DAOs, 15 Swing UI frames and dialogs, 24 utility/store/test classes, 2 root entry/connection classes, and 1 integration test harness):

Architectural Tier	Key Classes	Responsibilities
1. Presentation Layer (com.qveto.ecommerce.ui)	LoginFrame, DashboardFrame, ProductDetailsDialog, AddressBookDialog, OrderDetailsDialog, OrderTrackingDialog, OrderReturnDialog, ProductReviewDialog, NotificationCenterDialog, LowStockProcurementDialog, CategoryManagementDialog, EditProductDialog, UserDetailsDialog, AuditLogDialog, ProjectOverviewDialog	15 Swing frames and modal dialogs handling user interaction, tables, form validation, and live UI updates.
2. Service & Utility Layer (com.qveto.ecommerce.util)	UserStore, ProductStore, OrderStore, CategoryStore, CartStore, PaymentStore, CouponStore, ReviewStore, WishlistStore, NotificationStore, AddressBookStore, AuditStore, PdfInvoiceGenerator, ReportExportUtil	Coordinates business rules, coupon discounts, PDF/CSV exports, notifications, and in-memory caching.
3. Data Access Layer (com.qveto.ecommerce.dao)	UserDAO, CategoryDAO, ProductDAO, OrderDAO, OrderItemDAO, PaymentDAO, CartDAO, ReportDAO	Executes parameterized JDBC queries, joins, multi-table transactions, and analytical aggregations.
4. Persistence & Connection Layer (com.qveto.ecommerce & util)	DBConnection, DBConnectionPool, DatabaseRetryPolicy, DBStatusUtil, MySQL 8.x (schema + migrations)	Manages connection pooling, exponential backoff retries, health detection, and offline fallback.

2.2 Dual-Engine Execution Strategy

A core reliability requirement for evaluation is that the application never crashes on a machine where MySQL is absent or credentials are unconfigured:

- **Enterprise Live Mode:** DBConnection checks classpath and disk paths (src/main/resources/db.properties) for valid MySQL JDBC URL, username, and password settings.
- **Standalone Demo Mode:** If MySQL credentials are unset or the server is unreachable, DBStatusUtil detects the offline state cleanly and service stores serve consistent, pre-seeded in-memory data.
- **Live Health Indicator:** Both LoginFrame and DashboardFrame display a clear connectivity badge ('MySQL Active' vs 'Standalone Demo') so evaluators always know which engine is active.

Chapter 3: Technology Stack & Environment

The application deliberately minimizes third-party runtime dependencies so it compiles and executes on standard Java 17 or Java 21 installations.

Area	Technology / Component	Usage in Project
Programming Language	Java (JDK 17+ / JDK 21)	Core OOP modeling, business rules, concurrency, and Swing GUI
Desktop GUI Framework	Java Swing & AWT (javax.swing)	Native Look & Feel, JTables, custom dialogs, and GridBag/Border layouts
Typography & Rendering	High-DPI Antialiasing	Main.java sets awt.useSystemAAFontSettings=on and swing.aatext=true
Relational Database	MySQL 8.x	7 normalized tables, foreign keys, check constraints, and 6 reporting views
Database Connectivity	Java JDBC + mysql-connector-j 8.3.0	PreparedStatement queries and atomic commit/rollback transactions
Connection Pooling	Custom DBConnectionPool	Thread-safe blocking queue using java.lang.reflect.Proxy recycling
Fault Tolerance	DatabaseRetryPolicy	Exponential backoff retry loop with jitter for transient SQL exceptions
Document Generation	Pure Java Adobe PDF 1.4	Vector PDF tax invoice builder (PdfInvoiceGenerator) using Java streams
Data Exports	RFC 4180 CSV Engine	Exports product catalogs, master orders, sales reports, and audit logs
Security & Hashing	SHA-256 + DataSanitizer	Cryptographic password digests (PasswordUtil) and HTML/XSS neutralization
Build & Run Tools	Apache Maven & Batch Scripts	pom.xml build metadata plus 1-click run.bat and test.bat Windows launchers

Chapter 4: Customer Portal Modules

4.1 Authentication & Fast Demo Login (LoginFrame)

LoginFrame provides Customer Login, Admin Login, and Customer Registration tabs with password show/hide toggles and input validation. For evaluator convenience, both login tabs include '1-Click Fast Login' buttons to sign in immediately as the demo customer (customer@qveto.com) or administrator (admin), plus a shortcut to open ProjectOverviewDialog.

4.2 Catalog Discovery, Reviews & Wishlist

- **Catalog Table:** Customers can search products by keyword, filter by category, hide out-of-stock SKUs, and sort by price, product name, or stock level.
- **Product Details:** ProductDetailsDialog displays product descriptions, stock availability, live customer ratings, and verified buyer comments.
- **Saved Wishlist:** WishlistStore maintains saved customer items in a dedicated Wishlist tab with one-click 'Save to Wishlist' and 'Move to Cart' actions.

4.3 Shopping Cart & Promotional Coupon Engine

CartStore and CartDAO track selected items, enforce stock bounds, and synchronize quantity updates. During cart checkout, CouponStore validates promotional codes with minimum order thresholds and discount caps:

Promo Code	Discount Type	Min Order	Max Cap	Description
QVETO10	Percentage (10%)	$20.00	$50.00 cap	10% Off All Store Orders
SAVE20	Percentage (20%)	$80.00	$40.00 cap	20% Off Orders Above $80
FLAT15	Flat ($15.00)	$60.00	$15.00	$15 Instant Flat Discount
WELCOME	Percentage (15%)	$30.00	$25.00 cap	15% Welcome Customer Discount
MEGA50	Flat ($50.00)	$200.00	$50.00	$50 Off Bulk Orders Above $200

4.4 Saved Multi-Address Book & Smart Checkout

AddressBookStore and AddressBookDialog allow customers to maintain multiple delivery addresses tagged Home, Work, or Other and mark a default destination. At checkout, the address picker pre-populates the customer's default address and lets users switch locations or add a new address inline. PaymentStore processes simulated settlements across Cash on Delivery (COD), UPI, Credit/Debit Card, and Net Banking, producing TXN reference codes while reducing inventory stock.

4.5 Order Stepper, Vector PDF Invoices & Shipment Tracking

- **Order Stepper:** OrderDetailsDialog renders a 4-stage visual progression stepper (Placed -> Confirmed -> Shipped -> Delivered) alongside itemized prices, taxes, and applied promo discounts.
- **Vector PDF Invoices:** PdfInvoiceGenerator builds valid multi-section Adobe PDF 1.4 binary streams directly in Java (navy header bar, metadata box, shipping banner, itemized table, 5% tax calculation, grand total bar, and transaction verification box) without external JAR dependencies.

- **Live Shipment Tracking:** OrderTrackingDialog assigns courier carriers (BlueDart Express Priority or FedEx Air Logistics), generates a deterministic AWB tracking code (BD-EXP-...), and renders time-stamped transit checkpoints.

4.6 Post-Delivery Reviews, Returns & Notifications

Once orders reach DELIVERED status, customers can submit verified 1-to-5 star reviews via ProductReviewDialog (updating live product rating averages) or request returns with structured reason categories via OrderReturnDialog. NotificationCenterDialog displays unread store alerts (delivery updates, restocks, return approvals, and voucher notifications) via a live header alert counter.

Chapter 5: Administrator Command Center Modules

5.1 Product Catalog, Categories & Low-Stock Procurement

- **Catalog & Categories:** Administrators create, edit, restock, and remove products (EditProductDialog) and manage categories (CategoryManagementDialog) with foreign-key checks preventing deletion of categories still referenced by products.
- **Procurement Hub:** LowStockProcurementDialog monitors SKUs where stock_quantity <= low_stock_threshold, calculates total deficit exposure, and enables one-click single-SKU restocks or batch replenishment across multiple low items.

5.2 Order Fulfillment, Return Approvals & CSV Exports

Administrators manage all store orders in DashboardFrame, progressing statuses across PENDING, CONFIRMED, SHIPPED, and DELIVERED. When a customer submits a return request, OrderStore.approveReturn executes a single-click workflow that marks the order refunded, issues payment credit, and automatically adds the returned units back to product stock. Both ProductStore.exportCatalogCsv and OrderStore.exportOrdersCsv generate formatted RFC 4180 CSV files.

5.3 Executive Analytics, Customer Dossiers & Audit Journal

- **KPI Analytics:** The Reports & KPIs tab calculates six core metrics—Gross Revenue, Settled Collections, Total Orders, Average Order Value (AOV), Active Customers, and Low-Stock Exposure—with date-range filters (All Time, Last 7 Days, Last 30 Days, This Month) and best-seller rankings.
- **Customer Dossiers:** UserDetailsDialog shows user contact metadata, order counts, and Customer Lifetime Value (CLV), while preventing admins from deleting active administrator accounts accidentally.
- **Audit Journal:** AuditStore and AuditLogDialog maintain a timestamped audit trail capturing startup events, inventory restocks, order status changes, checkouts, coupon usage, and return approvals, with category filtering, keyword search, and CSV export.

Chapter 6: Application User Interface Showcase

The screenshots below capture the running Java Swing desktop application in Standalone Demo mode across authentication, customer shopping, administrator inventory control, and built-in review documentation:

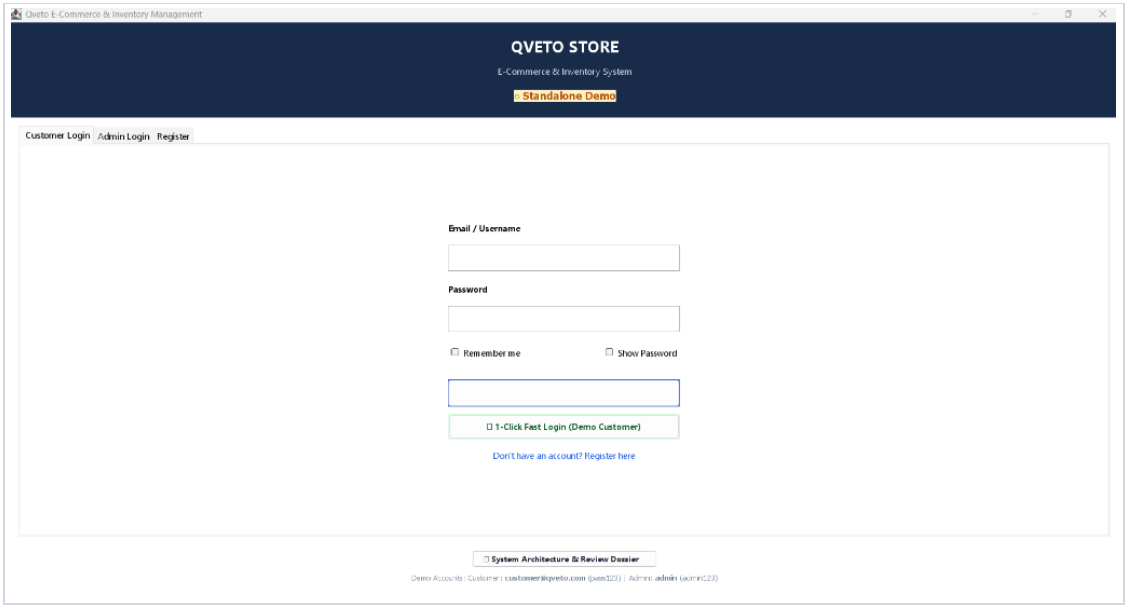


Figure 6.1 — Authentication Portal (LoginFrame): Shows role-separated Customer Login, Admin Login, and Register tabs alongside the Standalone Demo status badge, 1-Click Fast Login shortcuts, and the Review Dossier launcher.

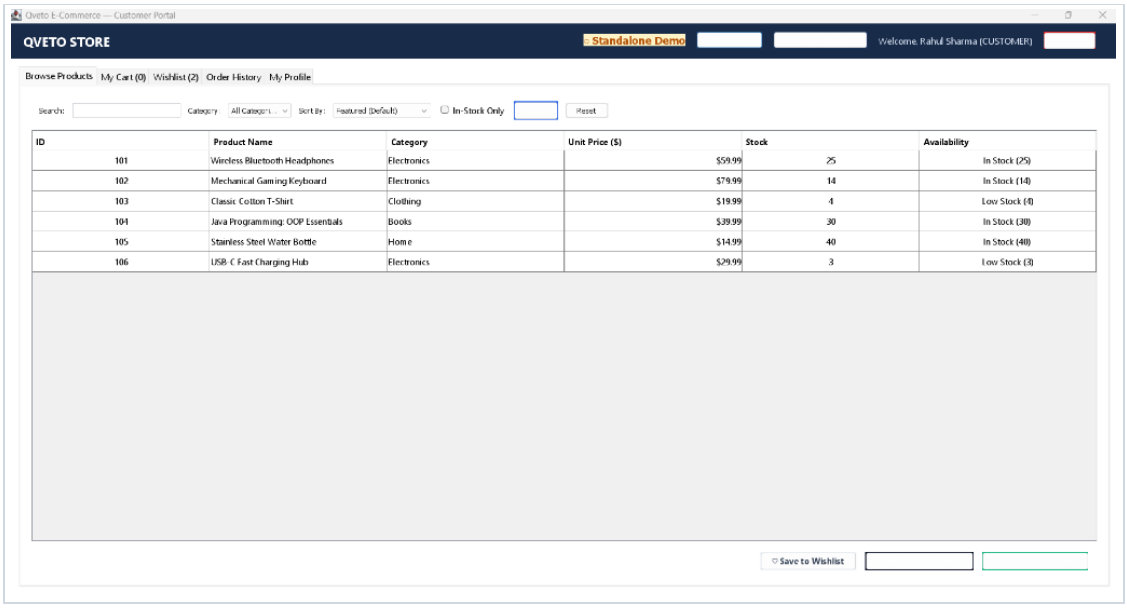


Figure 6.2 — Customer Storefront (DashboardFrame): Illustrates live keyword search, category filter, sort order dropdown, in-stock toggle, catalog table, and quick navigation tabs for Cart, Wishlist, Order History, and Profile.

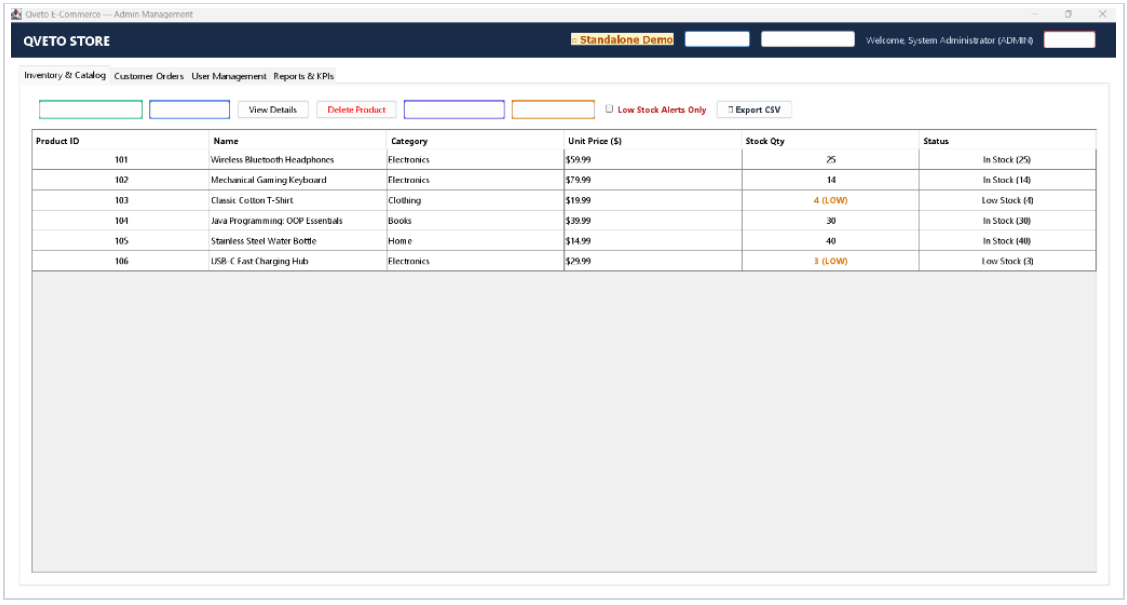


Figure 6.3 — Administrator Management Console (DashboardFrame): Presents unified management tabs (Inventory & Catalog, Customer Orders, User Management, Reports & KPIs) with highlighted LOW stock warnings for quick procurement restocking.

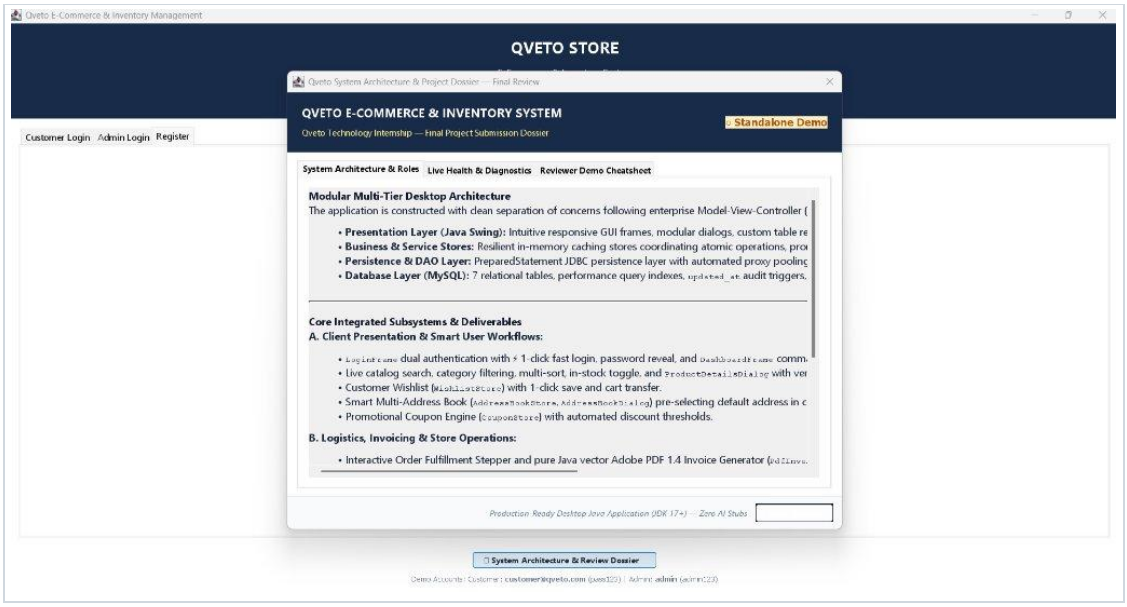


Figure 6.4 — System Architecture & Project Dossier (ProjectOverviewDialog): Displays the modal architecture dossier launched directly from LoginFrame, summarizing the four tiers, system health diagnostics, and evaluator credentials.

Chapter 7: Database Design & Persistence Infrastructure

7.1 Canonical Relational Schema (database/schema.sql)

Table Name	Primary Key	Purpose & Key Constraints
users	user_id (PK)	Stores CUSTOMER and ADMIN accounts, unique email, SHA-256 password_hash, phone, and address.
categories	category_id (PK)	Normalized product category names with UNIQUE constraints.
products	product_id (PK)	Stores SKU details, category_id (FK), price >= 0, stock_quantity >= 0, and low_stock_threshold >= 0.
cart_items	cart_item_id (PK)	Links user_id (FK) and product_id (FK) with quantity > 0 and UNIQUE (user_id, product_id).
orders	order_id (PK)	Order headers referencing users, ENUM status, payment_method, shipping_address, and total_amount >= 0.
order_items	order_item_id (PK)	Order line items referencing orders and products with quantity > 0 and unit_price >= 0 snapshot.
payments	payment_id (PK)	Tracks payment settlement records linked via order_id (FK), method, status, and amount >= 0.

7.2 Database Migrations (migration_v2.sql & migration_v3.sql)

- **Migration v2:** Adds 7 B-tree performance indexes (idx_products_category, idx_products_price, idx_cart_user_product, idx_orders_user, idx_orders_status, idx_order_items_order, idx_order_items_product) plus an updated_at TIMESTAMP column with ON UPDATE CURRENT_TIMESTAMP on the orders table.
- **Migration v3:** Defines 6 SQL analytical reporting views: view_sales_summary, view_top_selling_products, view_low_stock_alerts, view_category_sales_summary, view_customer_order_history, and view_payment_method_distribution.

7.3 Connection Pooling & Retry Policy

DBConnectionPool manages an ArrayBlockingQueue of JDBC connections (default initialSize = 3, maxSize = 10, timeout = 5000ms). Leased connections are wrapped via java.lang.reflect.Proxy so calling connection.close() returns the socket back to the idle queue rather than tearing down the TCP connection. Before each lease, connections are verified via isValid(2). DatabaseRetryPolicy classifies MySQL vendor error codes (such as 1205 lock wait timeout, 1213 deadlock, 2002/2003/2006/2013 connection faults) and SQLStates (08S01, 40001, etc.) as transient, retrying up to 3 attempts using exponential backoff (150ms initial, 2000ms max, 25% jitter), while immediately propagating fatal constraint or syntax errors.

Chapter 8: Security Architecture & Input Validation

- **Cryptographic Password Hashing:** PasswordUtil computes 64-character lowercase SHA-256 hexadecimal digests via java.security.MessageDigest matching MySQL's SHA2(password, 256) function, with verification fallback compatibility for seeded demo records.
- **SQL Injection Prevention:** Every DAO class uses parameterized JDBC PreparedStatement queries to neutralize SQL injection risks. DataSanitizer also includes defensive SQL injection pattern auditing.
- **XSS & Markup Sanitization:** DataSanitizer strips <script> blocks and HTML tags, escapes HTML entities (&, <, >, ", '), normalizes whitespace, strips non-printable control characters, and truncates strings safely to schema bounds.
- **Domain Boundary Validation:** InputBoundaryValidator and ValidationUtil enforce domain bounds on email formatting, password lengths, non-negative prices, and integer stock quantities before data reaches persistence.

Chapter 9: Testing & Quality Verification

To confirm correctness independently of manual GUI clicking, the repository provides two executable Java test suites compiled and launched together via test.bat. All 19 checks pass with zero failures on Adoptium JDK 21.

9.1 Suite 1: DatabaseIntegrityTest (8 / 8 Passed)

ID	Test Description	Result
TEST 1	PasswordUtil SHA-256 Hashing & Verification	PASSED
TEST 2	Model Financial Precision (BigDecimal Arithmetic)	PASSED
TEST 3	Inventory Stock Consistency & Restoration	PASSED
TEST 4	Category Foreign-Key Protection & Safe Deletion	PASSED
TEST 5	ReportDAO KPI Mathematical Consistency	PASSED
TEST 6	DatabaseRetryPolicy Transient Error Classification & Exponential Backoff	PASSED
TEST 7	DataSanitizer XSS Neutralization & Normalization	PASSED
TEST 8	InputBoundaryValidator Domain Entity Constraint Rules	PASSED

9.2 Suite 2: StoreWorkflowIntegrationTest (11 / 11 Passed)

ID	End-to-End Store Workflow Verified	Result
WF 1	Coupon Engine Discount Thresholds	PASSED
WF 2	Product Catalog & CSV Export	PASSED
WF 3	Order Lifecycle & Orders Master CSV	PASSED
WF 4	Executive Date-Range Sales Reporting	PASSED
WF 5	PDF 1.4 Vector Invoice Binary Generation	PASSED

ID	End-to-End Store Workflow Verified	Result
WF 6	Customer Verified Reviews & Star Ratings	PASSED
WF 7	Customer Wishlist & Saved Items	PASSED
WF 8	Order Return / Refund & Stock Replenishment	PASSED
WF 9	Store Notifications & Broadcast Hub	PASSED
WF 10	Customer Address Book & Multi-Destination Management	PASSED
WF 11	Store Audit Activity Journal & CSV Compliance Trail	PASSED

Chapter 10: Installation & Execution Guide

10.1 One-Click Launchers (Windows)

Located in C:\Users\annar\.gemini\antigravity\scratch\Qveto-java-team-project:

- **run.bat:** Compiles all packages into bin/ and starts com.qveto.ecommerce.Main with JDK 21 auto-discovery.
- **test.bat:** Compiles both main sources and test sources, then executes DatabaseIntegrityTest followed by StoreWorkflowIntegrationTest.

10.2 Standard Command Line Compilation & Execution

```
# 1. Compile all source and test files (Java 17+ / JDK 21)
javac -d bin src/main/java/com/qveto/ecommerce/model/*.java \
             src/main/java/com/qveto/ecommerce/dao/*.java \
             src/main/java/com/qveto/ecommerce/util/*.java \
             src/main/java/com/qveto/ecommerce/ui/*.java \
             src/main/java/com/qveto/ecommerce/*.java \
             src/test/java/com/qveto/ecommerce/*.java

# 2. Launch the Desktop Application (Windows classpath separator ';')
java -cp "bin;src/main/resources" com.qveto.ecommerce.Main

# 3. Execute Automated Test Harnesses
java -cp "bin;src/main/resources" com.qveto.ecommerce.util.DatabaseIntegrityTest
java -cp "bin;src/main/resources" com.qveto.ecommerce.StoreWorkflowIntegrationTest
```

10.3 Optional MySQL Setup & Built-in Demo Credentials

To connect to a local MySQL 8.x instance, run database/schema.sql, database/migration_v2.sql, database/migration_v3.sql, and database/seed.sql, and configure your credentials in src/main/resources/db.properties. Otherwise, run directly in Standalone Demo mode using the accounts below:

Role	Login ID / Email	Password	Accessible Capabilities
Customer	customer@qveto.com	pass123	Browsing, wishlist, coupons, address book, checkout, PDF invoice, AWB tracking, returns, reviews
Administrator	admin (seed: admin@qveto.com)	admin123	Inventory CRUD, category management, restock hub, order updates, return approvals, KPIs, audit journal

Chapter 11: Conclusion & Future Scope

The E-Commerce & Inventory Management System fulfills all functional and architectural goals defined for Project 1. By combining modular object-oriented Java design, a normalized MySQL schema with migrations and views, custom proxy-based connection pooling, zero-dependency vector PDF generation, and offline fallback resilience, the platform operates reliably in both production-style database setups and live demonstration environments.

Potential Future Extensions

- Extend inventory tracking across multiple regional warehouses with inter-branch stock transfers.
- Integrate live SMTP email dispatching so PDF invoices and tracking AWB updates are mailed automatically.
- Expose existing store and DAO layers through a lightweight REST API to support companion mobile or web storefronts.
- Add granular staff permissions (e.g., Warehouse Manager vs Finance Analyst) alongside the existing ADMIN role.